

HƯỚNG DẪN PHÁT TRIỂN BACK-END (BE) – AUTOWASH PRO

I. CẤU TRÚC PHÂN TẦNG TIÊU CHUẨN (SPRING BOOT 3)

Mã nguồn Back-end bắt buộc tổ chức theo kiến trúc phân tầng (Layered Architecture) với luồng phụ thuộc một chiều đi từ ngoài vào trong:

- config: Cấu hình hệ thống (Security, CORS, Swagger, Mapper...)
- controller: Tiếp nhận HTTP Request, gọi Service (Tầng Presentation)
- service: Tầng Business Logic (Chứa Service Interfaces)
- service/impl: Chứa các lớp Service Implementation (ServiceImpl)
- repository: Tầng Data Access - Giao tiếp với Database (Spring Data JPA)
- entity: Các thực thể cơ sở dữ liệu (Database Entities)
- dto: Lớp vận chuyển dữ liệu (Data Transfer Objects)
- dto/request: Chứa các DTO nhận từ client (ví dụ: CreateUserRequest)
- dto/response: Chứa các DTO trả về cho client (ví dụ: UserResponse)
- mapper: Ánh xạ tự động giữa DTO <-> Entity (ModelMapper / MapStruct)
- exception: Định nghĩa các Exception tùy chỉnh và Bộ xử lý lỗi tập trung
- exception/custom: Các Exception tự định nghĩa (ResourceNotFoundException...)
- exception/handler: Lớp GlobalExceptionHandler sử dụng @RestControllerAdvice
- utils: Các lớp tiện ích chung (String helper, Date helper...)

Nguyên tắc phụ thuộc: Luồng dữ liệu chỉ đi từ Controller -> Service Interface -> ServiceImpl -> Repository -> Database. Tuyệt đối không gọi ngược lại hoặc gọi nhảy cóc.

Cô lập Entity: Tầng Controller chỉ làm việc với các class trong gói dto. Không nhận hoặc trả trực tiếp Entity ra ngoài API để tránh rò rỉ cấu trúc DB.

II. THIẾT KẾ CODE & ĐƠN NHIỆM (SRP & SOLID)

- Giảm Liên Kết (Low Coupling): Tầng Controller chỉ được tham chiếu tới Service Interface. Sử dụng Dependency Injection (DI) thông qua @Autowired trên constructor.
- Không viết code get/set thủ công: Sử dụng thư viện ModelMapper hoặc MapStruct trong gói mapper để ánh xạ tự động giữa DTO và Entity.
- Đơn nhiệm (Single Responsibility Principle):
 - Controller: Chỉ nhận Request, kích hoạt Validation đầu vào (@Valid), gọi Service và trả về HTTP Status phù hợp. Không viết logic nghiệp vụ tại đây.

- **ServiceImpl:** Chứa toàn bộ logic nghiệp vụ của dự án. Ném ra các `RuntimeException` tùy chỉnh khi có lỗi nghiệp vụ xảy ra.
- **Repository:** Chỉ đảm nhận nhiệm vụ truy vấn dữ liệu từ DB.

III. VALIDATION & XỬ LÝ LỖI TẬP TRUNG (GLOBAL EXCEPTION HANDLER)

- **Validation Đầu Vào:** Sử dụng Jakarta Validation để kiểm tra tính hợp lệ ngay tại DTO: `@NotBlank`, `@NotNull`, `@Size`, `@Email`, `@Min`, `@Max`. Bắt buộc thêm annotation `@Valid` trước `@RequestBody` trong tham số của Controller.
- **Bộ Xử Lý Lỗi Tập Trung:** Nghiêm cấm lạm dụng khối lệnh try-catch lồng nhau.
- Định nghĩa các lớp ngoại lệ riêng kế thừa từ `RuntimeException` đặt tại `exception.custom` (ví dụ: `ResourceNotFoundException`, `BadRequestException`).
- Sử dụng `@RestControllerAdvice` để định dạng dữ liệu trả về cho client một cách thống nhất. Lỗi validation trả về 400 Bad Request. Các Custom Exception trả về mã HTTP tương ứng. Lỗi chung trả về 500 Internal Server Error kèm thông điệp thân thiện với người dùng (ẩn stack trace chi tiết).

IV. QUY TẮC RESTFUL API & DATABASE DESIGN

- **Đặt Tên RESTful API:**
- Sử dụng danh từ số nhiều đại diện cho tài nguyên trong URI (ví dụ: `/api/v1/users`). Không sử dụng động từ.
- Áp dụng đúng HTTP Methods (GET, POST, PUT, PATCH, DELETE).
- Sử dụng chữ thường và dấu gạch nối (kebab-case) cho URI. Luôn có tiền tố phiên bản `/api/v1/`.
- **Thiết Kế Database (MS SQL Server):**
- **Tên bảng:** Chữ thường, danh từ số nhiều, sử dụng `snake_case` (ví dụ: `users`, `product_categories`).
- **Tên cột:** Chữ thường, `snake_case`.
- **Khóa chính:** Luôn đặt tên cột là `id`. Khóa ngoại: Đặt tên theo định dạng `tên_bảng_số_ít_id` (ví dụ: `user_id`).
- Sử dụng rõ ràng các annotation `@Table(name = "...")` và `@Column(name = "...")` trong Entity mapping.

V. TÀI LIỆU HÓA SWAGGER / OPENAPI

- Tích hợp dependency `springdoc-openapi-starter-webmvc-ui` phiên bản mới nhất vào `pom.xml`.
- Mô tả API bằng Annotations:
- Sử dụng `@Tag(name = "...", description = "...")` ở cấp Controller để phân nhóm API.

- Sử dụng `@Operation(summary = "...", description = "...")` ở cấp endpoint để mô tả chức năng của API.
- Sử dụng `@Schema(description = "...", example = "...")` ở cấp thuộc tính của DTO để mô tả dữ liệu và cung cấp ví dụ mẫu.
 - Địa chỉ xem tài liệu Swagger: <http://localhost:8080/swagger-ui/index.html> sau khi chạy ứng dụng.

VI. CẤU HÌNH RUNTIME & DATABASE CHUNG

- Môi trường chạy mặc định: Java 17 (LTS) & Maven.
- Cấu hình Database local chung (MS SQL Server):
- Port kết nối: 1433
- Tài khoản: sa
- Mật khẩu: 123456
- Tên database: autowash_pro
 - Quy trình thiết lập DB: Khi bắt đầu phiên làm việc thiết lập môi trường, Agent sẽ hỏi lập trình viên: "Bạn có muốn sử dụng Docker để chạy Database SQL Server không? (Có / Không)"
- Nếu Chọn Có (Docker): Hướng dẫn chạy lệnh docker compose up -d ở thư mục gốc dự án để khởi động container autowash-sqlserver.
- Nếu Chọn Không (Cài thủ công): Hướng dẫn cài đặt SQL Server Developer/Express, SSMS, bật Mixed Mode, cấu hình TCP/IP Port 1433.

VII. QUY TẮC PHỐI HỢP & NHẬT KÝ LÀM VIỆC (JOURNALING)

- Xác nhận danh tính: Agent hỏi tên của lập trình viên 1 lần duy nhất khi clone dự án về máy lần đầu.
- Nếu tên là Anh (Reviewer): Nhật ký làm việc được cập nhật trực tiếp vào file cập nhật chung.
- Nếu tên khác (ví dụ: Binh, Cuong...): Tạo file nhật ký riêng dạng journal-[tên].md trong thư mục `.agents/learnings/`. Tuyệt đối không ghi đè hoặc chỉnh sửa nhật ký của người khác.